



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

Modelo de Classes - CSU05

Designer de Sistemas

Equipe 03 - Jogo Desconecta

Ana Lúcia de Souza - 5084

Florestal

2025

Sumário

CSU05: Calcular Pontuação.....	3
Back-end.....	3
Classes de Entidade:.....	4
Classes de Controle:.....	5
Classes de acesso ao Banco de Dados:.....	5
Classe auxiliar: PontuacaoService:.....	5
Relacionamento entre as classes.....	6
Front-end.....	6

CSU05: Calcular Pontuação

Back-end

A figura 1 mostra o Diagrama de Classes de considerável parte do sistema codificado até aqui, faltando apenas o jogo Conecta, o jogo Caça-palavras, a interface que unirá os dois jogos ao restante do desafio e o ranking. Neste documento, vamos nos concentrar nas partes desse diagrama que se referem ao cálculo da pontuação referente a cada jogo, ficando, portanto, o cálculo da pontuação total do aluno como uma responsabilidade do CSU08: Apresentar Classificação. A figura 2 demonstra o diagrama para o CSU05.

Você pode acessar o diagrama abaixo através de seu e-mail institucional clicando [aqui](#).

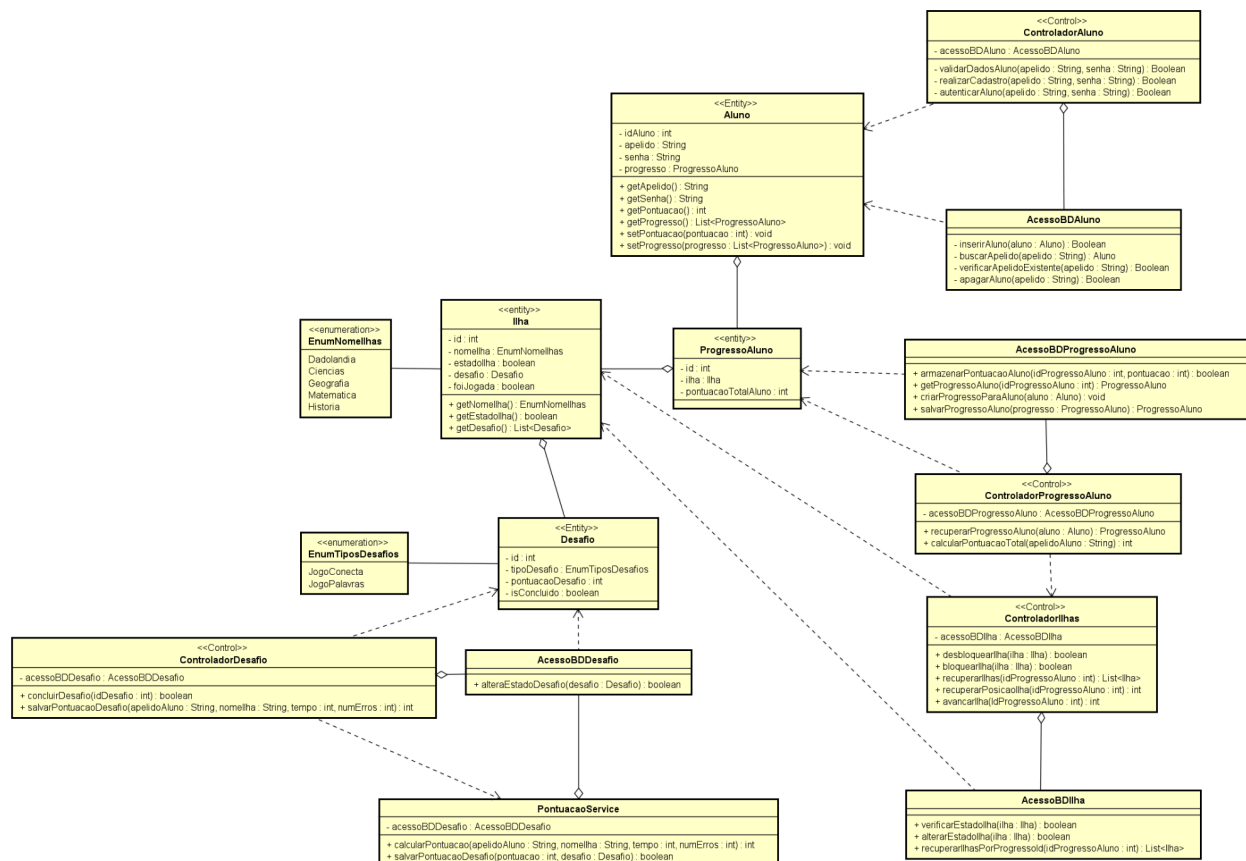


Figura 1. Diagrama de Classes parcial Desconecta.

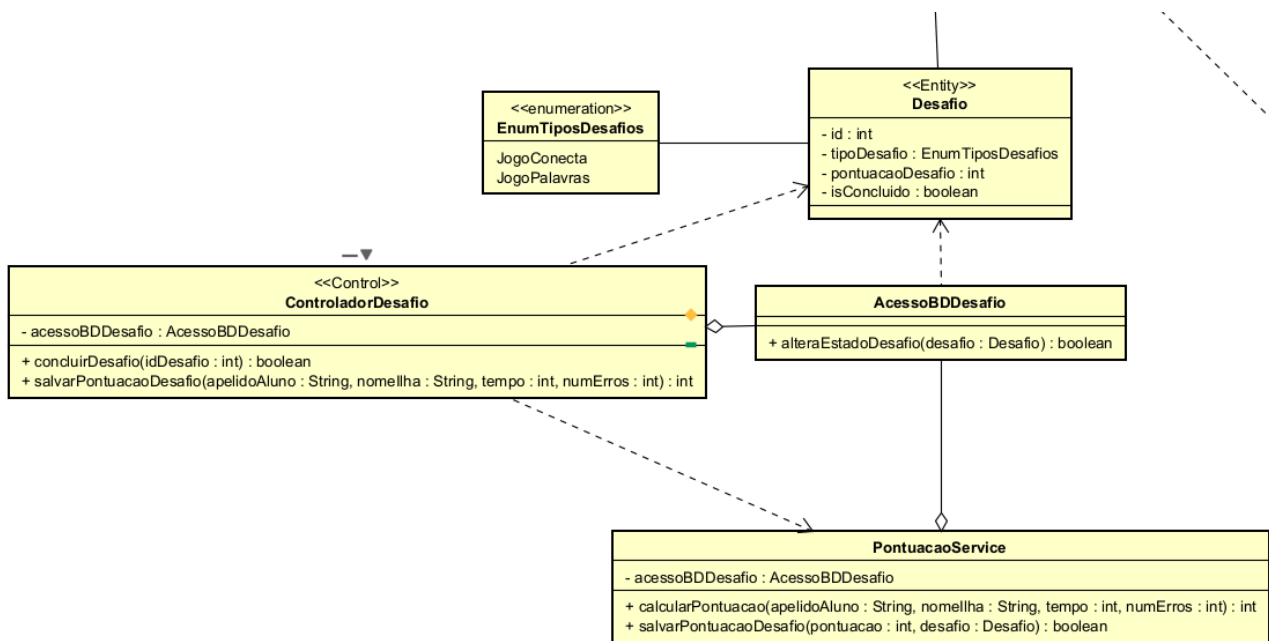


Figura 2. Diagrama de Classes para calcular pontuação.

Classes de Entidade:

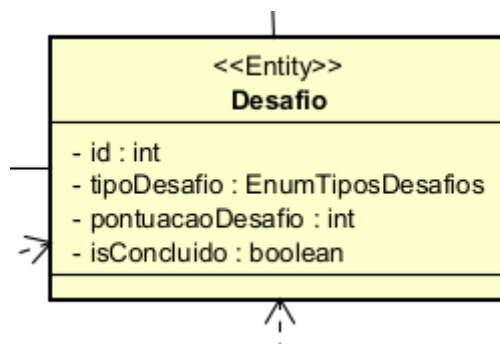


Figura 3. Classe Desafio.

- **Desafio:** Essa classe possui os seguintes atributos privados.
 - **idDesafio:** é o identificador do jogo, ou seja, representa ou um Conecta ou um Caça-palavras.
 - **TipoDesafio:** é um Enum que tem por objetivo representar se um jogo é um Conecta ou um Caça-palavras.
 - **PontuacaoDesafio:** Guarda o valor da pontuação obtido pelo aluno em cada desafio (Conecta ou Caça-palavras).
 - **IsConcluido:** Tem por objetivo guardar se o jogo em questão foi concluído pelo aluno ou não. Esse atributo só deve ser marcado como verdadeiro após

tudo der certo no cálculo da pontuação, ou seja, se algo der errado com os dados necessários para o cálculo, o aluno deve poder jogar o desafio novamente, para que sua pontuação seja contabilizada.

- Os métodos getter e setters podem ser implementados conforme a necessidade do programador.

Classes de Controle:

A classe “**ControladorDesafio**”, possui o atributo privado **acessoBDDesafio**, que será criado pelo construtor dessa classe. Nesta classe existem os seguintes métodos:

- **concluirDesafio()**: Esse método tem como objetivo marcar o desafio como concluído para que, caso o aluno o jogue novamente sua nova pontuação não seja contabilizada, ou seja, a pontuação do aluno deve ser contada apenas na primeira vez que o mesmo joga um desafio.
- **salvarPontuacaoDesafio()**: Esse método irá receber do front-end os dados necessários para o cálculo da pontuação. Em seguida, esse método deve verificar se esses dados não estão vazios e chamar **calcularPontuacao()** da classe **pontuacaoService**, para que o cálculo possa ser realizado. Se houver problemas com os dados necessários para a pontuação, deve-se reportar um erro e solicitar ao aluno, via front-end, para que jogue o jogo novamente. Após o cálculo, o método **guardarPontuacaoDesafio()** deve ser chamado para que a pontuação seja persistida no banco de dados e o método **concluirDesafio()** deve ser usado para marcar o desafio como concluído. Por fim, o método **salvarPontuacaoDesafio()** deve retornar a pontuação ao front-end.

Classes de acesso ao Banco de Dados:

A classe **AcessoBDDesafio** será a responsável pelo acesso e manipulação do banco de dados referente à tabela **Desafio**. Seu método será descrito a seguir:

- **alterarEstadoDesafio()**: Esse método tem por objetivo alterar o estado de um desafio (marcado como concluído ou como ainda não jogado).

Classe auxiliar: **PontuacaoService**:

- **calcularPontuacao()**: Esse método será o responsável por calcular a pontuação de um desafio, não importando se ele é um Conectar ou um Caça-palavras, ou seja, o cálculo do desafio deve ser feito da mesma forma para ambos, usando portanto, os mesmos parâmetros e a mesma fórmula de cálculo.

- **guardarPontuacaoDesafio():** Esse método será o responsável por salvar, no banco de dados, a pontuação referente ao desafio jogado pelo aluno.

Relacionamento entre as classes

Observe que a classe **ControladorDesafio** apresenta um relacionamento de agregação com **AcessoBDDesafio**, sendo a primeira o objeto "todo" e a segunda o objeto "parte". O mesmo ocorre entre **PontuacaoService** e **AcessoBDDesafio**. A agregação representa que o controlador "tem um" acesso ao banco de dados para cumprir sua função. Em relação a **ControladorDesafio**, há uma relação de dependência com as classes **Desafio** e **PontuacaoService**. Em relação a **AcessoBDDesafio** há uma relação de dependência com a classe **Desafio**.

Front-end

Para a modelagem do front-end, foi utilizado o diagrama de máquina de estados conforme apresentado nas figuras 4 e 5. Nesse diagrama, **os estados representam as telas ou popUps** e **as transições representam os botões**.

Observe que os diagramas das figuras 4 e 5 representam Casos de uso já implementados. Observe que eles já contemplam a pontuação que foi chamada de "TelaParabéns". Assim, toda vez que um dos jogos termina, através do "BotaoVerificar", a tela de pontuação (TelaParabens) é exibida contendo a pontuação referente ao jogo em questão.

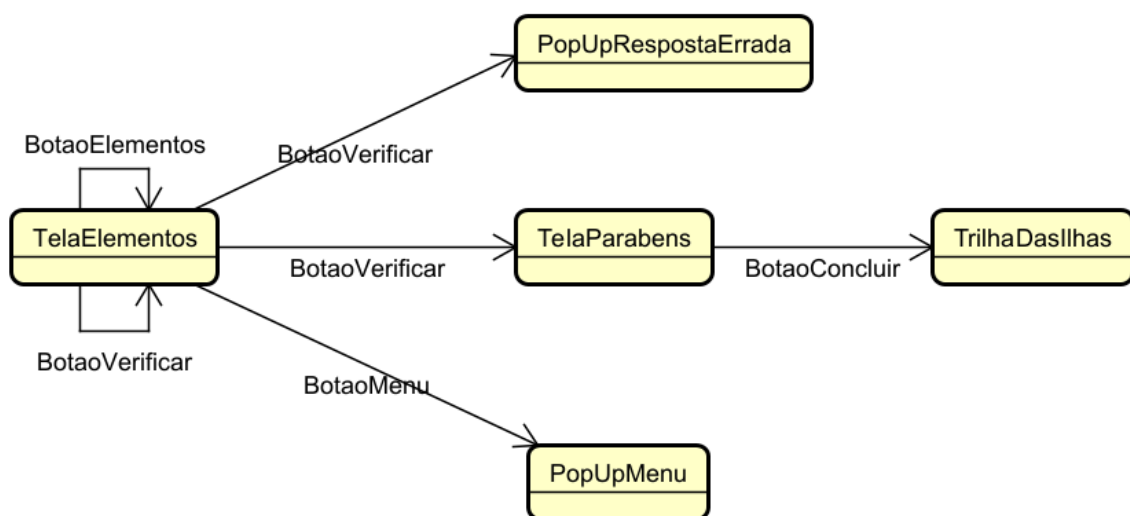


Figura 4. Modelagem front-end para CSU06 contendo a TelaParabens (Pontuação).

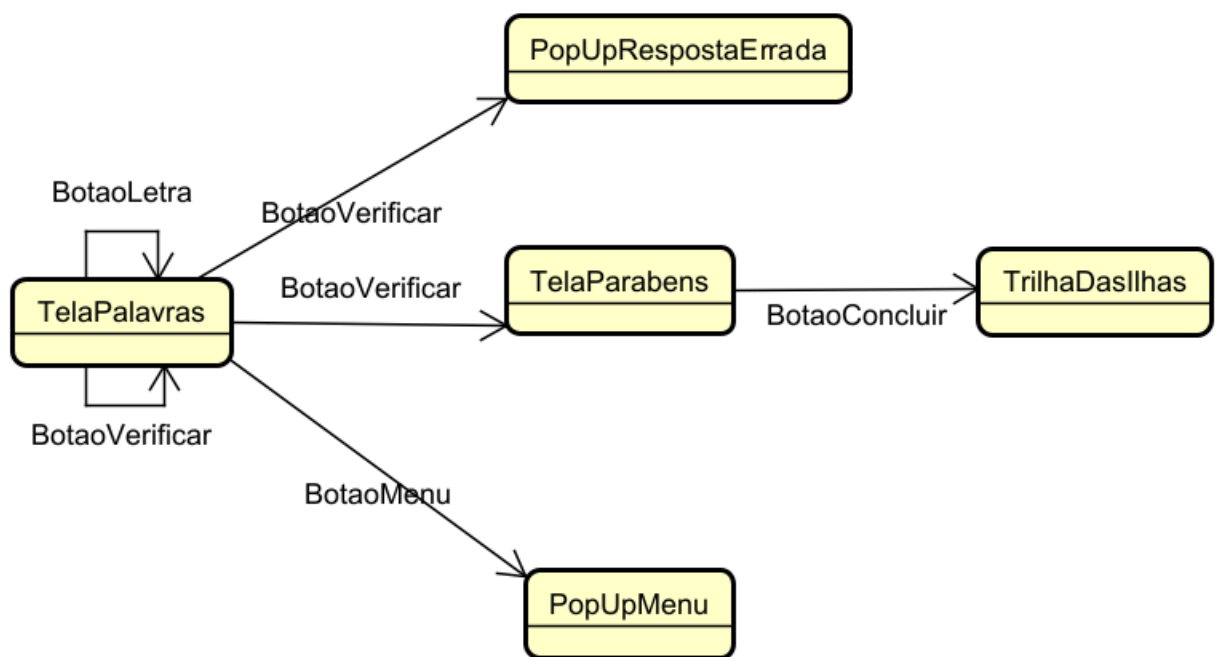


Figura 5. Modelagem front-end para CSU07 contendo a TelaParabens (Pontuação).



Figura 6. Prototipagem de TelaParabens.